

Name of the Student	D.Chinna Moulali
Reg. No	192425278
GitHub Link	https://github.com/kiduuuu/MLA0405-Deep-learning

SIMATS ENGINEERING

CO4 - ASSESSMENT TOOL 1

Design and Develop Model

COURSE NAME: Deep Learning
SLOT : D

COURSE CODE: MLA0405

COURSE OUTCOME COVERED

CO 4: Students will be able to understand and apply probabilistic graphical models including Bayesian Networks, Markov Random Fields, Hidden Markov Models, and Conditional Random Fields. They will learn how to represent complex probabilistic relationships among variables and perform inference and learning in graphical models. Students will be able to use these models for reasoning under uncertainty and solving real-world decision-making problems. BL-6

Course Outcome Weightage: 10%

Duration: 90 minutes

Assessment Tool Weightage: 30%

Mark : 25

Code of Conduct:

I D.Chinna Moulali (192425278) that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: *D. Chinna Moulali*

Name of the student: D.Chinna Moulali

Criteria	Problem Understanding & Requirement Analysis (2)	Model Architecture & Design (2.5)	Application of Deep Learning Techniques (2.5)	Model Development & Implementation Strategy (2)	Performance Analysis & Justification (1)	Total (10)
Weightage	20 %	30%	25%	15 %	10 %	100 %
Q1						
Q2						
Q3						
Q4						
Q5						
Total						

QUESTIONS:

05

Total Marks:25

1. Design and develop a model for a hospital has only 2,000 labeled X-ray images for classifying images into normal and abnormal categories. A pre-trained ResNet model is available. Design a CNN-based transfer learning model by identifying which layers should be frozen, which layer should be replaced, and how the model should be trained.

Solution:

CNN-BASED TRANSFER LEARNING FOR HOSPITAL X-RAY CLASSIFICATION

1. Problem Understanding and Requirement Analysis

The hospital has only 2,000 labeled X-ray images, which is a relatively small dataset for training a deep CNN from scratch. The objective is to classify X-ray images into two categories:

- **Normal**
- **Abnormal**

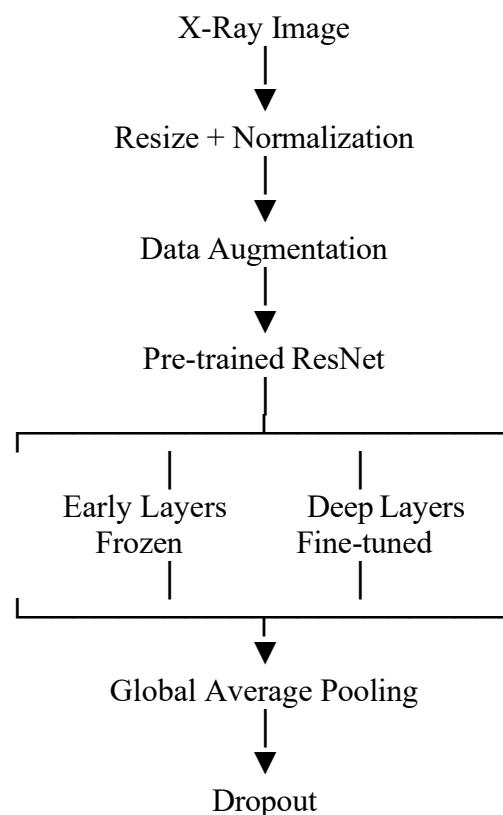
Training a CNN from scratch may result in overfitting because millions of parameters need to be learned from only 2,000 images. Therefore, transfer learning using a pre-trained ResNet model is an appropriate solution.

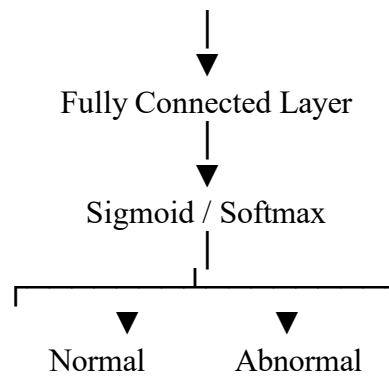
A suitable dataset can be divided into:

- 70% training
- 15% validation
- 15% testing

Data augmentation should be used to increase the effective diversity of the training data.

2. Proposed Architecture





3. Frozen Layers

The early convolutional layers of ResNet should initially be frozen.

These layers learn general visual features such as:

- Edges
- Lines
- Shapes
- Textures
- Basic patterns

These features are useful even for medical X-ray images.

The deeper layers learn more task-specific features and can therefore be fine-tuned.

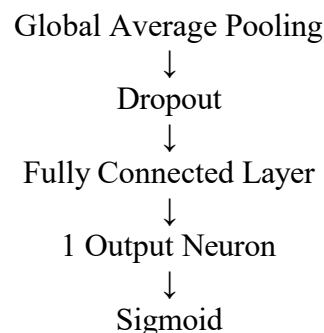
A practical strategy is:

ResNet Early Layers	→ Frozen
ResNet Middle Layers	→ Frozen initially
ResNet Deep Layers	→ Fine-tuned
Original FC Layer	→ Removed
New Classification Head	→ Trainable

4. Replacing the Final Layer

The original ResNet classification layer is designed for its original training classes, so it should be removed.

It can be replaced with:



For binary classification:

Output > 0.5 → Abnormal

Output ≤ 0.5 → Normal

Alternatively, two output neurons with Softmax can be used.

5. Training Strategy

The training can be performed in two stages.

Stage 1 – Train Classification Head

Freeze the ResNet backbone and train only the new classification layer.

Frozen ResNet

+

Trainable Classifier

This allows the new classifier to learn the relationship between ResNet features and normal/abnormal X-rays.

Stage 2 – Fine-Tuning

Unfreeze the deeper ResNet layers and train them using a small learning rate.

This allows the network to adapt generic image features to medical X-ray patterns.

6. Data Preprocessing

The X-ray images can undergo:

- Resizing
- Pixel normalization
- Small rotations
- Cropping
- Scaling
- Horizontal flipping where clinically appropriate

Care must be taken with medical images because some augmentations may change clinically meaningful information.

Class imbalance can be handled using:

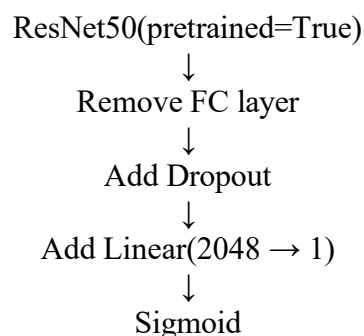
- Class-weighted loss
- Oversampling
- Focal loss

7. Tools and Implementation

Recommended tools:

- Python
- PyTorch or TensorFlow
- torchvision
- OpenCV
- NumPy
- Pandas
- Matplotlib
- GPU/CUDA

Example model concept:



8. Performance Analysis

Since this is a medical application, accuracy alone is insufficient.

The model should be evaluated using:

- Accuracy
- Precision
- Recall/Sensitivity
- Specificity
- F1-score
- ROC-AUC
- Confusion matrix

Sensitivity is particularly important because incorrectly classifying an abnormal X-ray as normal could be clinically serious. A separate test set must be used for final evaluation to avoid data leakage.

9. Justification

Transfer learning is suitable because only 2,000 labeled images are available.

It provides:

- Faster training
- Better feature extraction
- Reduced overfitting
- Lower computational requirements
- Better generalization than training from scratch

Therefore, a pre-trained ResNet with frozen early layers, fine-tuned deeper layers, and a new binary classification head is a practical solution for the hospital.

2. An agricultural organization wants to identify five types of plant diseases from leaf images. Since the dataset contains limited training samples, the organization wants to reuse features efficiently. Design a DenseNet-based classification model and explain how dense connections can help the model reuse features during classification.

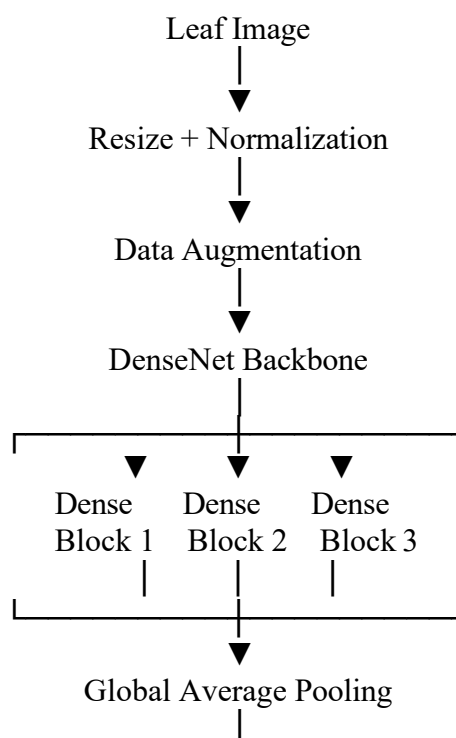
Solution:

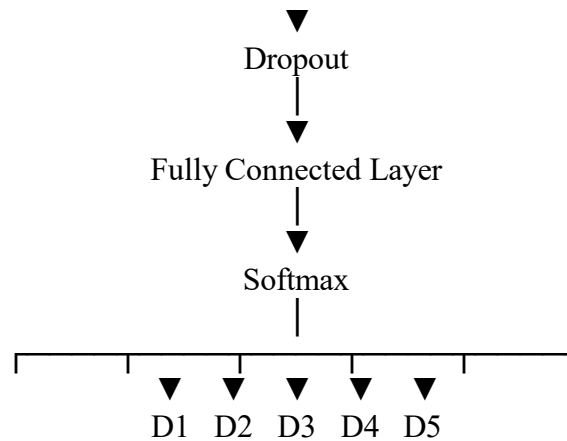
DENSENET-BASED PLANT DISEASE CLASSIFICATION

1. Problem Understanding

An agricultural organization wants to identify five plant diseases using leaf images. The dataset contains limited training samples. Therefore, the model must learn useful features efficiently while minimizing unnecessary feature learning. A DenseNet-based classification model is suitable because DenseNet encourages extensive feature reuse through dense connections.

2. Proposed Architecture



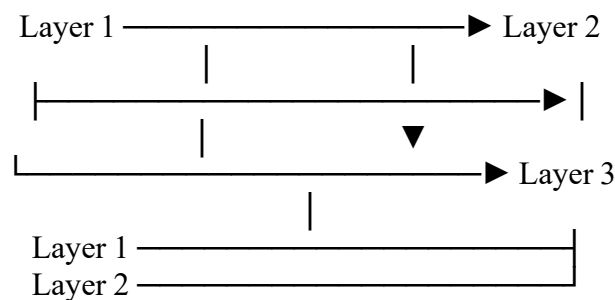


Where D1–D5 represent the five plant diseases.

3. Dense Connections

The major feature of DenseNet is that each layer receives feature maps from all preceding layers within a dense block.

Conceptually:



Instead of:

Layer 1 → Layer 2 → Layer 3 → Layer 4

DenseNet uses:

Layer 1 → Layer 2

Layer 1 → Layer 3

Layer 2 → Layer 3

Layer 1 → Layer 4

Layer 2 → Layer 4

Layer 3 → Layer 4

4. Feature Reuse

Earlier layers may learn:

- Leaf edges
- Veins
- Color patterns
- Textures

Later layers may learn:

- Spots
- Lesions
- Disease patterns
- Shape abnormalities

Dense connections allow later layers to directly access earlier features.

Therefore, the model does not need to relearn the same features repeatedly.

5. Transfer Learning Strategy

Because the dataset is limited, a pre-trained DenseNet such as DenseNet121 can be used.

Initially:

DenseNet Backbone → Frozen
Classification Head → Trainable
After initial training:
Last Dense Block → Fine-tuned
Classification Head → Trainable
A small learning rate should be used during fine-tuning.

6. Training

Data augmentation can include:

- Rotation
- Scaling
- Cropping
- Horizontal flipping
- Brightness adjustment

The model can use categorical cross-entropy loss:

$$\text{Loss} = -\sum y_i \log(p_i)$$

The optimizer can be:

- Adam
- AdamW
- SGD with momentum

7. Tools

Implementation can use:

- Python
- PyTorch/TensorFlow
- torchvision
- OpenCV
- NumPy
- Matplotlib
- GPU/CUDA

A suitable agricultural dataset could contain labeled leaf images for the five disease classes.

8. Performance Analysis

Evaluation metrics include:

- Accuracy
- Precision
- Recall
- F1-score
- Confusion matrix
- Macro-F1

A confusion matrix is useful for identifying which diseases are frequently confused with one another.

9. Justification

DenseNet is particularly suitable for a limited dataset because its dense connections:

- Improve feature reuse
- Improve gradient flow
- Reduce redundant feature learning
- Encourage efficient representation learning
- Help the network learn both low-level and high-level disease features

Therefore, a pre-trained DenseNet121 with dense feature reuse and fine-tuning provides an efficient solution for five-class plant disease classification.

3. A university wants to predict a student's future academic performance using their weekly attendance, assignment marks, internal assessment marks, and previous examination scores collected over several weeks. Design an LSTM-based model that can process this sequential information and predict the student's performance category.

Solution:

LSTM-BASED STUDENT ACADEMIC PERFORMANCE PREDICTION

1. Problem Understanding

A university wants to predict a student's future academic performance using information collected over several weeks.

The available sequential features are:

- Weekly attendance
- Assignment marks
- Internal assessment marks
- Previous examination scores

The important point is that the data is sequential.

For example:

Week 1 → Week 2 → Week 3 → Week 4 → ... → Week N

The performance in one week may influence the interpretation of later performance.

Therefore, an LSTM network is suitable.

2. Input Representation

For each week, create a feature vector:

Week t = [Attendance, Assignment, Internal, Exam Score]

For example:

Week 1 = [90, 75, 70, 68]

Week 2 = [92, 78, 73, 72]

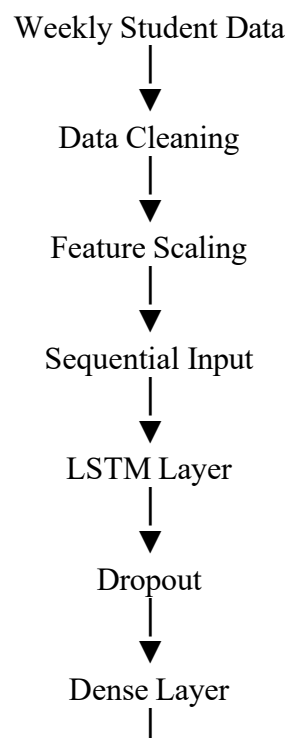
Week 3 = [85, 70, 68, 70]

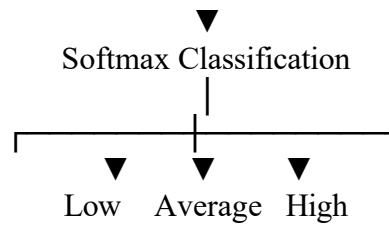
...

The input therefore has the shape:

Number of Students $\times$ Number of Weeks $\times$ 4 Features

3. Proposed Architecture

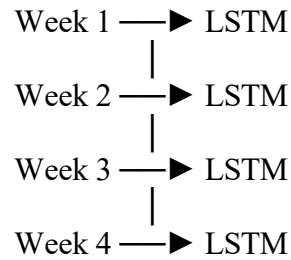




4. Role of LSTM

LSTM maintains information from previous time steps.

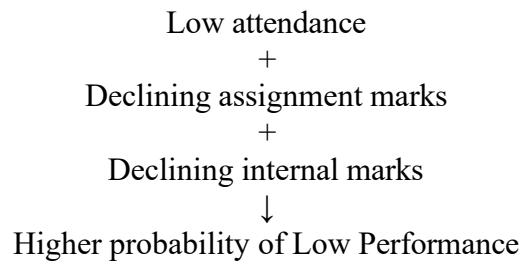
For example:



Student Pattern

Future Performance

The model can learn patterns such as:



5. LSTM Components

LSTM uses gates to control information.

Forget Gate

Determines which previous information should be discarded.

Input Gate

Determines which new information should be stored.

Cell State

Maintains important information across time.

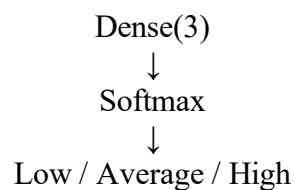
Output Gate

Determines the information passed to the next layer.

This makes LSTM more suitable than a basic RNN for sequential academic data.

6. Output

The final layer can contain three neurons:



Example:

Low = 0.10

Average = 0.25

High = 0.65

Prediction:

High Performance

7. Training Strategy

The dataset should be divided chronologically or by student to prevent leakage.

Possible split:

Training → 70%

Validation → 15%

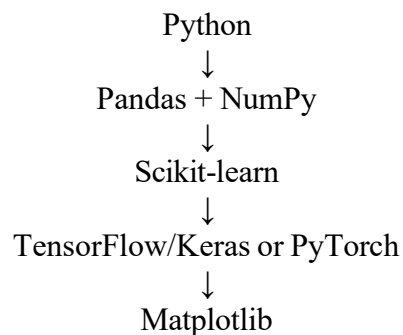
Testing → 15%

Features should be normalized using training-set statistics.

The model can use:

- Categorical Cross-Entropy
- Adam optimizer
- Early stopping
- Dropout

8. Tools



9. Performance Analysis

The model can be evaluated using:

- Accuracy
- Precision
- Recall
- F1-score
- Confusion matrix

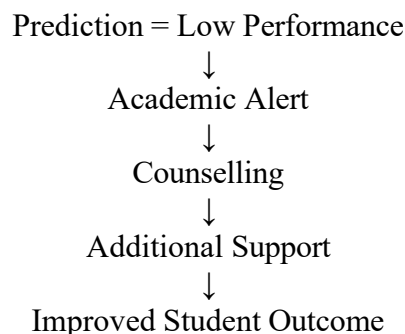
If the classes are imbalanced, macro-F1 should also be reported.

The university can use the model to identify students who may require academic support.

10. Industry/Institutional Benefit

The system can provide early intervention.

For example:



Thus, LSTM transforms weekly academic records into a sequential representation and predicts the student's future performance category.

4. An organization wants to classify customer reviews as positive, negative, or neutral. Since the meaning of a word may depend on both the words appearing before and after it, design a Bidirectional RNN model for this application. Identify the major processing stages from input text to final sentiment prediction.

Solution:

BIDIRECTIONAL RNN FOR CUSTOMER SENTIMENT CLASSIFICATION

1. Problem Understanding

An organization wants to classify customer reviews into:

- Positive
- Negative
- Neutral

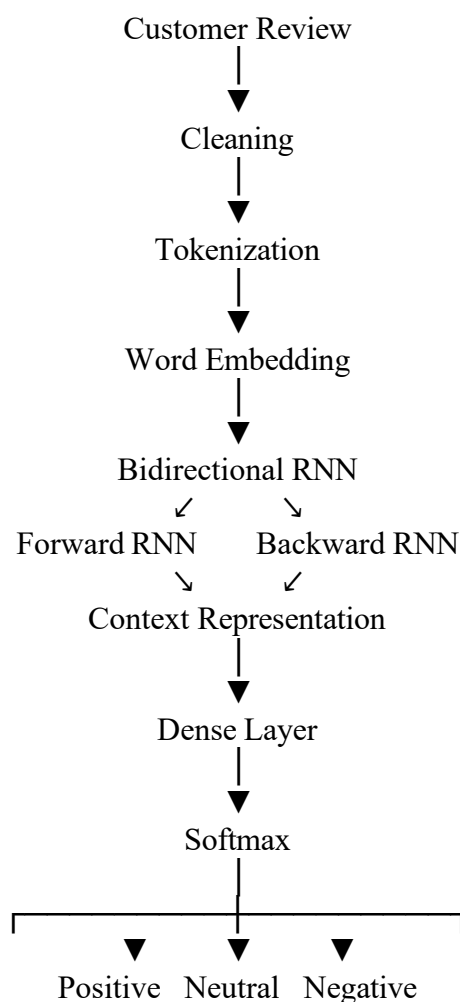
The meaning of a word can depend on both its previous and subsequent words.

For example:

"The product is not good."

The word "good" alone appears positive, but the word "not" changes the overall meaning. Therefore, a Bidirectional RNN is suitable because it processes text from both directions.

2. Proposed Architecture



3. Major Processing Stages

Stage 1 – Text Collection

Collect customer reviews from:

- Product reviews
- Surveys
- Customer feedback
- Social platforms

Stage 2 – Text Preprocessing

Perform:

- Lowercasing
- Tokenization
- Handling punctuation
- Removing unnecessary symbols
- Padding sequences

Stage 3 – Word Embedding

Words are converted into numerical vectors.

"excellent" → [0.23, 0.81, ...]

"bad" → [0.17, 0.12, ...]

Embeddings allow the model to learn semantic relationships between words.

Stage 4 – Bidirectional RNN

The sentence is processed in two directions.

Forward:

The → product → is → not → good

Backward:

good → not → is → product → The

The two hidden representations are combined.

$h_{\text{forward}} + h_{\text{backward}}$
↓
Combined Context

Stage 5 – Classification

The combined representation is passed to a dense layer.

Bidirectional RNN
↓
Dense Layer
↓
Softmax

The output contains three probabilities.

Example:

Positive = 0.12

Neutral = 0.08

Negative = 0.80

Final prediction:

Negative

4. Why Bidirectional RNN?

A normal RNN mainly processes information in one direction.

A Bidirectional RNN captures:

- Previous context
- Future context
- Word relationships
- Sentence structure
- Context-dependent meaning

This is particularly useful for sentiment classification.

5. Training

The model can use:

Loss Function → Categorical Cross-Entropy

Optimizer → Adam

Activation → Softmax

Training data:

Review + Sentiment Label

Example:

"The product is excellent" → Positive

"The product is okay" → Neutral

"The product is terrible" → Negative

6. Tools

Recommended tools:

- Python
- TensorFlow/Keras or PyTorch
- NLTK / spaCy
- Pandas
- NumPy
- Matplotlib

Suitable datasets include:

- IMDb-derived sentiment datasets
- Amazon review datasets
- Twitter sentiment datasets

7. Performance Analysis

The model should be evaluated using:

- Accuracy
- Precision
- Recall
- F1-score
- Confusion matrix

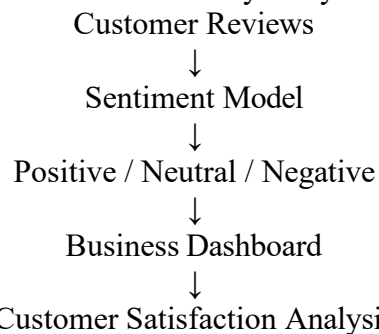
Because there are three classes, macro-F1 is useful when class distribution is uneven.

Error analysis should specifically examine:

- Negation
- Sarcasm
- Mixed sentiment
- Very short reviews
- Domain-specific words

8. Industry Benefit

The organization can use the model to automatically analyze thousands of customer reviews.



This helps businesses identify customer dissatisfaction and improve products and services.

5. A software company wants to develop a system that translates English sentences into Hindi . The length of the input and output sentences may differ, and the system needs to understand the complete input sentence before generating the translated sentence. Design an Encoder–Decoder model and describe the role of the encoder, decoder, hidden representation, and output layer in the translation process.

Solution:

ENCODER–DECODER MODEL FOR ENGLISH-TO-HINDI TRANSLATION

1. Problem Understanding

A software company wants to translate English sentences into Hindi.

The input and output sentences may have different lengths.

For example:

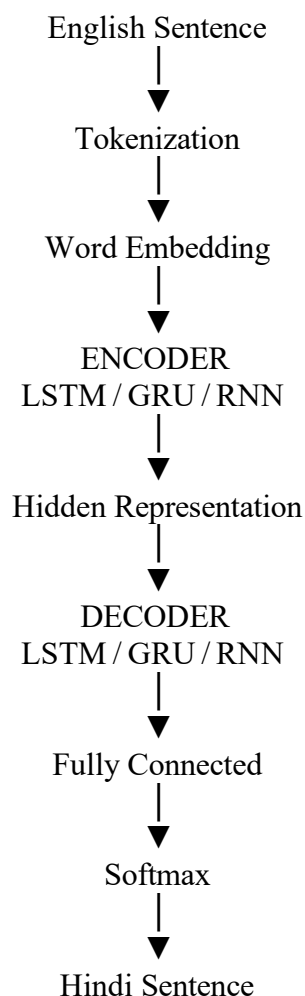
English:
"I am going to school."

Hindi:
"मैं स्कूल जा रहा हूँ।"

The model therefore needs to understand the complete source sentence before generating the target sentence.

An Encoder–Decoder sequence-to-sequence architecture is appropriate.

2. Proposed Architecture



3. Encoder

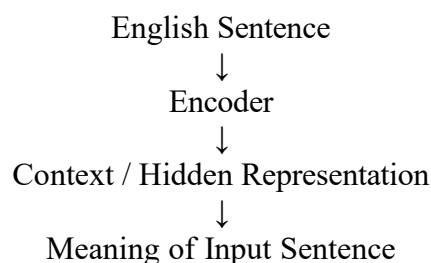
The encoder receives the English sentence one token at a time.

I → am → going → to → school

Each word is converted into an embedding vector.
The encoder processes these vectors sequentially and produces hidden states.
Its objective is to capture the meaning and context of the complete input sentence.

4. Hidden Representation

The encoder produces a representation containing information about the input sentence.
Conceptually:

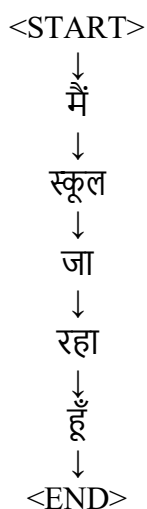


This representation is passed to the decoder.

For longer sentences, an attention mechanism should be added instead of relying only on a single fixed vector. Attention allows the decoder to focus on different parts of the English sentence while generating each Hindi word.

5. Decoder

The decoder generates the Hindi translation sequentially.
It starts with a special <START> token.

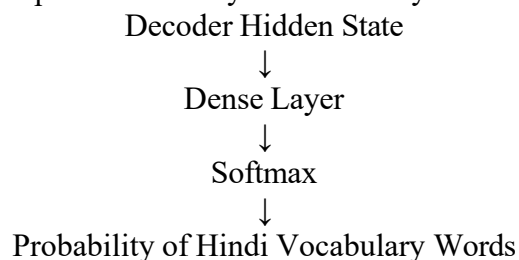


At each time step, the decoder predicts the next word using:

- Previous generated word
- Decoder hidden state
- Encoder information
- Attention context, if used

6. Output Layer

The decoder's hidden state is passed to a fully connected layer followed by Softmax.



For example:

मैं → 0.70

वह → 0.10
स्कूल → 0.05
घर → 0.03

...

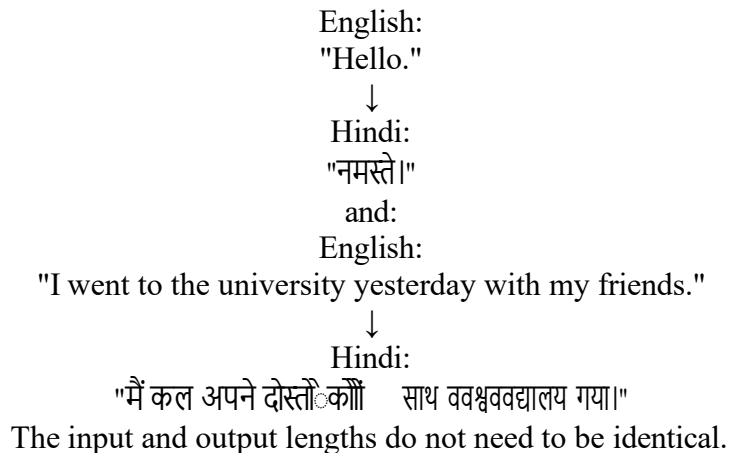
The highest-probability suitable word is selected.

The process continues until <END> is generated.

7. Handling Different Sentence Lengths

The Encoder–Decoder architecture naturally supports variable-length sequences.

For example:



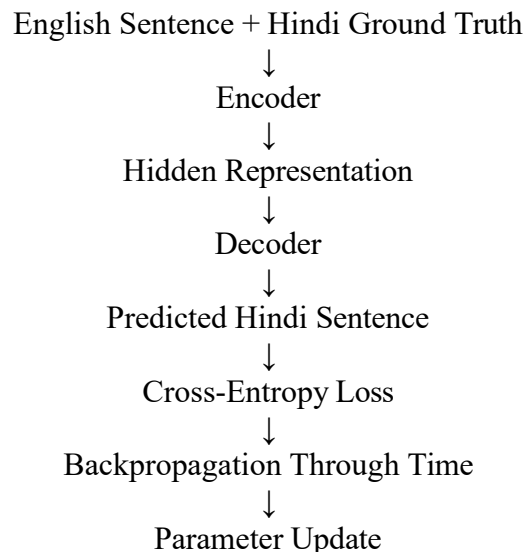
8. Training Strategy

The model can be trained using parallel English-Hindi sentence pairs.

Suitable resources include:

- OPUS
- IIT Bombay English-Hindi Corpus
- WMT-related translation datasets

Training process:



Teacher forcing can be used during training, where the correct previous target word is supplied to the decoder.

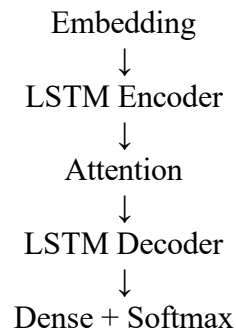
9. Tools and Implementation

Recommended tools:

- Python
- PyTorch / TensorFlow

- NumPy
- Pandas
- SentencePiece / tokenizers
- Hugging Face libraries
- CUDA/GPU

The model can use:



10. Performance Analysis

Translation quality can be measured using:

- BLEU
- METEOR
- ROUGE
- Human evaluation

The system should also be evaluated for:

- Grammar
- Context preservation
- Meaning preservation
- Handling of long sentences
- Proper nouns
- Different sentence structures

11. Why Encoder–Decoder is Suitable

The architecture separates the translation process into two major tasks:

Encoder → Understand the English sentence

Decoder → Generate the Hindi sentence

This makes it possible to handle:

- Different input/output lengths
- Sequential language generation
- Contextual relationships
- Variable sentence structures

Adding attention further improves translation because the decoder can dynamically access relevant encoder states.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding & Requirement Analysis	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding ; some requirements unclear	Poor understanding ; misinterprets problem context
Model Architecture & Design	30%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Application of Deep Learning Techniques	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Model Development & Implementation Strategy	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Performance Analysis & Justification	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis